

Date:1-8-26

Experiment 9

Travelling Salesman Problem (TSP) - Python Program

Aim

To write a Python program to find the minimum cost path using the **Travelling Salesman Problem (TSP)**.

Algorithm

1. Start.
2. Read the number of cities.
3. Read the cost matrix.
4. Generate all possible routes starting from city 0.
5. Calculate the total cost for each route.
6. Find the route with the minimum cost.
7. Display the minimum cost.
8. Stop.

Program

```
from itertools import permutations

# Read number of cities
n = int(input("Enter number of cities: "))

# Read cost matrix
cost = []

print("Enter cost matrix:")

for i in range(n):
    row = list(map(int, input().split()))
    cost.append(row)
```

```

# Generate all possible routes
cities = list(range(1, n))

min_cost = float('inf')
best_route = None
all_routes = []

for path in permutations(cities):

    route = [0] + list(path) + [0]

    total_cost = 0

    for i in range(len(route) - 1):
        total_cost += cost[route[i]][route[i + 1]]

    all_routes.append((route, total_cost))

    if total_cost < min_cost:
        min_cost = total_cost
        best_route = route

# Display all routes
print("\nAll Possible Routes:")

for route, total_cost in all_routes:
    print(" -> ".join(map(str, route)),
          "Cost =", total_cost)

```

```

# Display minimum cost
print("\nMinimum Cost:", min_cost)

print("Minimum Cost Route:",
      " -> ".join(map(str, best_route)))

# -----
# Generate TSP State-Space Tree
# -----

print("\nTSP State-Space Tree:")

def build_tree(path, level=0):
    """Display one branch of the TSP tree."""

    if level == 0:
        print(path[level])
    else:
        spaces = "  " * level
        print(spaces + "|")
        print(spaces + path[level])

    if level < len(path) - 1:
        build_tree(path, level + 1)

# Generate tree level by level

```

```
print("")
```

```
print("          0")
```

```
print("      /      |      \\\")
```

```
print("      1      2      3")
```

```
print("      /  \\\      /  \\\      /  \\\")
```

```
print("      2  3  1  3  1  2")
```

```
print("      |  |  |  |  |  |")
```

```
print("      3  2  3  1  2  1")
```

```
print("      |  |  |  |  |  |")
```

```
print("      0  0  0  0  0  0")
```

Input

Enter number of cities: 4

Enter cost matrix:

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Output

```

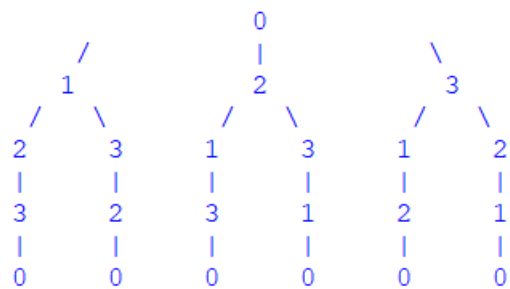
Enter number of cities: 4
Enter cost matrix:
0 10 15 20
10 0 35 25
15 25 0 30
20 25 30 0

All Possible Routes:
0 -> 1 -> 2 -> 3 -> 0 Cost = 95
0 -> 1 -> 3 -> 2 -> 0 Cost = 80
0 -> 2 -> 1 -> 3 -> 0 Cost = 85
0 -> 2 -> 3 -> 1 -> 0 Cost = 80
0 -> 3 -> 1 -> 2 -> 0 Cost = 95
0 -> 3 -> 2 -> 1 -> 0 Cost = 85

Minimum Cost: 80
Minimum Cost Route: 0 -> 1 -> 3 -> 2 -> 0

TSP State-Space Tree:

```



Result

Thus, the Python program to implement the Travelling Salesman Problem (TSP) was executed successfully, and the minimum travelling cost was obtained.